SPACE BAGUETTES • STUDENT HANDBOOK

GitHub for Our Team

Our repo: JennyZhang9/SpaceBaguette.github.io
https://github.com/JennyZhang9/SpaceBaguette.github.io

A step by step guide to setting up Git and sharing a project without overwriting your teammates' work. Use the setup pages once. Use the team workflow every time you start a new task.

Before you start

Use this guide on Windows or Mac. You need internet, an email address, and permission to install software. Everyone clones the same repo. Our shared branch is main.

Age rule: GitHub accounts require users to be at least 13. Some countries require an older age. If you are younger, do not create an account or share someone else's login. Work on local files and give them to your teacher to review and upload.



Words you will see in this guide

Word	What it means
Git	A program on your computer that tracks file changes.
GitHub	A website where the team stores and reviews its project.
Repository or repo	The project folder and its saved history.
Branch	A separate path for changes inside the same repo.
Commit	A saved checkpoint in Git on your computer.
Push and pull	Push uploads commits. Pull downloads and combines updates.
Pull request or PR	A request on GitHub to review and merge a branch.
Merge	Combine a reviewed branch into main.

Team rule: one branch per person per task. Edit on your branch, push it, and open a pull request. A teammate or teacher checks it before it joins main.

1 Create your account and join the team

Each student age 13 or older

1. Go to https://github.com and choose Sign up. Follow the prompts to create a free personal account. Use your own email and a strong password. Keep your password private.

2. Verify your email using the message GitHub sends. Complete any security setup GitHub asks for. Save recovery codes somewhere safe if you enable two factor authentication.

3. Send your GitHub username to your teacher. A username is different from your email address.

4. Accept the repository invitation from your email or GitHub notifications. Open https://github.com/JennyZhang9/SpaceBaguette.github.io while signed in. You need collaborator access to push a branch.

Team setup for Jenny

Our repository already exists: JennyZhang9/SpaceBaguette.github.io. Its default branch is main. Students should clone this repo; they do not need to create a new one.

In the repo, open Settings → Collaborators or the equivalent access page. Invite each eligible student by username with permission to contribute. Give everyone the same repo link. Students do not need to make separate repos or forks for this workflow.

When available, protect main with a branch rule that requires pull requests and review. Available protection options depend on the account plan and repository visibility. Agree that students will use pull requests even if a rule is unavailable.

Agree on tasks before editing

Student	Task	Example branch
Alex	Make a sleep plot	alex/sleep-plot
Sam	Build the dashboard	sam/dashboard
Riley	Write the project description	riley/readme

Try to work in different files. If two people need the same file, talk first. Branches help organize work, but they cannot prevent every conflict.

2 Install Git and set your name

A terminal is a window where you type commands. Type one command at a time and press Enter. Do not type a leading $ or PS prompt. Commands below are text you can copy.

Windows

1. Visit https://git-scm.com/install/windows and download Git for Windows for your computer. Run the installer. Keep recommended defaults, including Git available from the command line and Git Credential Manager. If asked to choose an editor, choose one you recognize, such as Notepad.

2. Close any old terminal windows. Open Git Bash from the Start menu. Use Git Bash for all commands in this guide.

Mac

1. Open Terminal from Applications → Utilities. Run the command below. If prompted, install the Command Line Tools and wait for installation to finish.

```
xcode-select --install
```

2. If it says the tools are already installed, continue. Other installation options are at https://git-scm.com/install/mac.

Check Git and identify your commits

```
git --version
git config --global user.name "Your Name"
git config --global user.email "YOUR_EMAIL"
```

Replace Your Name and YOUR_EMAIL with your own details, keeping the quotes. Use your verified GitHub email, or your no-reply email copied from GitHub Settings → Emails for privacy. These settings label your commits; they do not log you in.

Sign in when Git asks

On Windows, Git Credential Manager normally opens browser sign in when you first access a private repo or push. Sign in to the GitHub account invited to the team.

On Mac, if browser sign in is not available, have your teacher help install GitHub CLI from https://cli.github.com. Run gh auth login, select GitHub.com and HTTPS, agree to authenticate Git, and choose browser sign in.

Do not enter your normal GitHub password into a Git terminal password prompt. GitHub does not accept it for HTTPS Git operations. Ask your teacher if sign in gets stuck.

3 Clone the repo and open the folder

Copy the team repo address

1. Open https://github.com/JennyZhang9/SpaceBaguette.github.io. Click Code → HTTPS. The clone address is the exact address in the command below.

2. In Git Bash on Windows or Terminal on Mac, run these commands one at a time. This clone command is ready to copy exactly as written.

```
mkdir -p ~/GitProjects
cd ~/GitProjects
git clone https://github.com/JennyZhang9/SpaceBaguette.github.io.git
cd SpaceBaguette.github.io
git status
```

Git creates a folder named SpaceBaguette.github.io. The .git ending belongs to the clone address; it is not part of the folder name. Do not add .git to the cd command.

Success check: git status should show the branch name and “working tree clean.” Clone once per computer. Do not run git init inside this folder. Download ZIP is not the same as cloning.

Open the correct project folder

In your editor, use File → Open Folder and select SpaceBaguette.github.io inside GitProjects in your home folder. Your home folder is usually C:\Users\yourname on Windows or /Users/yourname on Mac. Keep all team files inside the cloned repo.

Each time you reopen the terminal, return to the repo with the exact command below. Then run git status.

```
cd ~/GitProjects/SpaceBaguette.github.io
```

Find your project files

Actual file or folder	What to know
README.md	The team project description. Read it first.
index.html	The entry page at the top of the repo.
astra-pulse/	The website project folder.
astra-pulse/index.html	The website page inside that folder.
astra-pulse/app.js	A JavaScript file used by the website.

Open the whole SpaceBaguette.github.io folder in your editor. For website work, edit the files Jenny assigns inside astra-pulse. Keep your terminal at the repo’s top level so the commands in this guide use the correct paths.

Do not reorganize existing files or add a new folder system without checking with Jenny. Our repo is public, so keep passwords, tokens, and private health information out of it.

4 Use a branch for every new task

This example uses alex/sleep-plot. Replace alex with your name and sleep-plot with a short task name. Use a new branch name for each new task. First, finish and commit any current work so git status says “working tree clean.”

Check the branch name before editing. It should be your task branch, not main. Saving a file in your editor does not make a commit. A commit stays on your computer until you push it.

Command or action	What it does
`git status`	Check your current branch and unsaved Git changes.
`git switch main`	Go to the team’s local main branch.
`git pull --ff-only origin main`	Update main from GitHub. If it fails, stop and see section 6.
`git switch -c alex/sleep-plot`	Create your new branch and switch to it.
Edit and save files in your editor	For practice, create alex-practice.txt in the repo’s top folder and add one sentence. Jenny should approve practice PRs.
`git status` `git diff`	Check changed files and review edits before saving a commit.
`git add alex-practice.txt`	Stage this file. Use the paths of your actual changed files.
`git diff --staged`	Review exactly what will enter the next commit.
`git commit -m "Add practice note"`	Save a checkpoint. Use a message that describes your work.
`git push -u origin alex/sleep-plot`	Upload your branch. -u connects it to its GitHub branch.

Keep working on the same task

If you are already on this task branch, keep editing, saving, reviewing, staging, and committing. After its first push, use git push. More pushes to this branch update the same open pull request.

```
git status
git diff
git add alex-practice.txt
git diff --staged
git commit -m "Improve practice note"
git push
```

The filename here is an example. For several files, list their paths after git add, separated by spaces. Put quotes around paths containing spaces. Use git add . only after checking every changed file; it stages all changes under the current folder.

5 Review and merge on GitHub

Open a pull request

1. After pushing, open the repo on GitHub. Click Compare & pull request. If that button is missing, open Pull requests → New pull request.

2. Set base to main and compare to your branch, such as alex/sleep-plot. Base is the destination. Compare is the branch carrying your changes.

3. Add a clear title and a short explanation: what changed, how you tested it, and whether you need help. Click Create pull request.

4. Ask a teammate or teacher to review the changed files. The reviewer should check that the work makes sense and that the project still runs. Fix feedback on the same branch, then commit and push again.

5. After review and any required checks pass, a person with permission clicks Merge pull request and confirms. If GitHub reports conflicts, use section 6 first. Delete the remote task branch after merging if offered.

What happens when two students work

Stage	Alex	Sam	main on GitHub
Start	Creates alex/sleep-plot	Creates sam/dashboard	Shared starting version
Work	Commits and pushes a plot	Commits and pushes dashboard	Unchanged by their pushes
Review	Opens PR to main	Opens PR to main	Reviewer checks each PR
Combine	Alex’s PR merges first	Updates branch if needed, then merges PR	Now contains both changes

Push uploads your branch; it does not merge it into main. A pull request is different from git pull: the PR asks for review, while git pull downloads updates to your computer.

After your pull request is merged

Wait until GitHub shows that your PR is merged. With a clean working tree, update your local main. Start a fresh branch for your next task.

```
git switch main
git pull --ff-only origin main
git switch -c alex/next-task
```

Bring team updates into your branch

Commit your current edits first. While on your task branch, run these commands to bring the latest main into your branch. If a conflict appears, stop and follow section 6. Otherwise push the updated branch.

```
git fetch origin
git merge --no-edit origin/main
git push
```

These commands update your task branch. They do not merge your task into main. Keep one person responsible for each task branch. Sharing a branch requires extra coordination.

6 Fix common problems

If a merge reports a conflict

A conflict means Git needs help choosing the final content. It often happens when two people change the same lines. Your work is still there. Ask the other author or teacher to help decide what to keep.

1. Run git status to see the conflicted files. Open each one in your editor. A text conflict may look like this:

```
<<<<<<< HEAD
Your branch’s text
=======
The incoming text from main
>>>>>>> origin/main
```

2. Replace that whole block with the final text you both want. Keep either version, or combine them. Remove all marker lines. Save the file. For images or other non-text files, ask the teacher to help choose the correct file.

3. Test the result. Then stage every resolved file and finish the merge. Replace README.md with the actual file path. Push your task branch again.

```
git add README.md
git commit -m "Resolve merge conflict"
git push
```

To cancel an in-progress merge instead, use git merge --abort. This guide starts merges with a clean working tree. Do not use reset --hard or force push to fix a conflict.

Quick troubleshooting

Message or situation	Next step
`git is not recognized or command not found`	Reopen Git Bash or Terminal after installation. Check git --version.
Not a git repository	Run cd ~/GitProjects/SpaceBaguette.github.io to enter the repo. Run git status.
Repository not found or permission denied	Check the URL, accepted invitation, and signed-in GitHub account.
Nothing to commit	Save your file in the editor. Check git status and the folder you opened.
Local changes would be overwritten	Stay on your task branch. Review and commit your edits before switching or merging.
Push rejected as non-fast-forward	On your task branch with clean status, run git pull --no-rebase, resolve conflicts if needed, then git push. If the branch is shared, coordinate first.
Pull --ff-only fails on main	Local main and GitHub main may have different commits. Stop and ask the teacher to preserve your work and help move it to a task branch.
Branch already exists	Use git switch BRANCH_NAME to resume that task, or choose a new name for a new task.

7 Keep this command sheet nearby

Run commands inside the cloned repo. Replace example names and paths. Run git status often. Work on your task branch and keep main for reviewed work.

Check that you saved and tested your work, committed the correct files, pushed your task branch, and opened a PR with base main. Tell the reviewer what to check. Never push directly to main as part of this team workflow.

When	Commands

Start a new task
Clean working tree first
	`git switch main`
	`git pull --ff-only origin main`
	`git switch -c alex/task-name`

Save a checkpoint
Save files in your editor first
	`git status`
	`git diff`
	`git add path/to/file`
	`git diff --staged`
	`git commit -m "Describe your change"`

Upload the branch the first time	`git push -u origin alex/task-name`
Upload later commits	`git push`

Get main’s updates on your task branch
Commit your edits first
	`git fetch origin`
	`git merge --no-edit origin/main`
	`git push`

Resume an existing local task branch Clean working tree first	`git switch alex/task-name` `git pull --no-rebase`
After your PR is merged Clean working tree first	`git switch main` `git pull --ff-only origin main`
See where you are	`git status` `git branch`

Official help and sources

Instructions checked October 12, 2026. Website button names may change. The links below provide current official help.

Account setup: https://docs.github.com/en/get-started/onboarding/getting-started-with-your-github-account

Account age rules: https://docs.github.com/en/site-policy/github-terms/github-terms-of-service

Git installers: https://git-scm.com/install

HTTPS sign in: https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git

Team workflow: https://docs.github.com/en/get-started/using-github/github-flow

Pull requests: https://docs.github.com/articles/creating-a-pull-request

Conflict help: https://docs.github.com/en/pull-requests/how-tos/merge-and-close-pull-requests/resolving-a-merge-conflict-using-the-command-line